

Сравнение и анализ оптимизаций компиляторов

Созонов Александр Сергеевич

Группа: 25.Б71-мм

Кафедра: Системное программирование

Научный руководитель: инженер-исследователь Романова Зинаида Андреевна

Номер семестра практики: 2

1. Постановка задачи

Работа выполняется в рамках учебной практики. **Целью работы является** проведение сравнительного анализа оптимизаций компиляторов с рассмотрением их влияния на производительность и размер исполняемого кода. Результаты работы необходимы для понимания оптимизаций, проводимых современными компиляторами. Их понимание позволит лучше понять, как именно происходит трансляция кода в инструкции процессора, что нужно для написания более эффективных и безопасных программ.

2. Описание предлагаемого решения

Для сравнения были выбраны 3 следующих компилятора:

- CLang (версия 22.1.1)
- MSVC (версия 19.50.35726)
- Intel oneAPI DPC++/C++ Compiler (версия 2025.3.2) (далее — ICX)

Характеристики системы, на которой проводились замеры времени работы:

- Процессор: AMD Ryzen 7730U with Radeon Graphics
- ОЗУ: 16 GB, 3200 MHz
- ОС: Windows 11 Pro 23H2
- x86-64 архитектура

Работа предполагает сравнение и анализ времени выполнения программы, её размера и оптимизаций, проводимых компиляторами. Для всего этого использовалась специальная программа-тест, включающая в себя арифметические действия, циклы, записи в массивы и прочее, что может быть оптимизировано. Анализ проводился над ассемблерным кодом, а замеры с помощью команды Measure-Command в PowerShell.

3. Замеры

CLang (без оптимизаций: clang -O0 optbench.c -o nooptimization.exe):

- Размер: 143 872 байт
- Среднее: 333,9534 мс
- Выборочное СКО: 9,8824 мс

CLang (с оптимизацией по скорости работы: clang -O2 optbench.c -o speedopt.exe):

- Размер: 144 384 байт
- Среднее: 6,9212 мс
- Выборочное СКО: 0,2602 мс

CLang (с оптимизацией по размеру файла: clang -Os optbench.c -o sizeopt.exe):

- Размер: 144 384 байт
- Среднее: 6,7660 мс
- Выборочное СКО: 0,3498 мс

MSVC (без оптимизаций: cl /Od optbench.c):

- Размер: 140 800 байт
- Среднее: 336,0474 мс
- Выборочное СКО: 13,7329 мс

MSVC (с оптимизацией по скорости работы: cl /O2 optbench.c):

- Размер: 139 776 байт
- Среднее: 7,1323 мс
- Выборочное СКО: 0,2715 мс

MSVC (с оптимизацией по размеру файла: cl /O1 optbench.c):

- Размер: 139 264 байт

- Среднее: 7,0232 мс
- Выборочное СКО: 0,3688 мс

ICX (без оптимизаций: icx /Od optbench.c):

- Размер: 140 288 байт
- Среднее: 332,6697 мс
- Выборочное СКО: 7,1367 мс

ICX (с оптимизацией по скорости работы: icx /O2 optbench.c):

- Размер: 140 800 байт
- Среднее: 6,8673 мс
- Выборочное СКО: 0,2711 мс

ICX (с оптимизацией по размеру файла: icx /Os optbench.c):

- Размер: 140 800 байт
- Среднее: 6,9220 мс
- Выборочное СКО: 0,2644 мс

Анализируя данные результаты можно сделать следующие первичные выводы: все компиляторы так или иначе хорошо справляются с оптимизациями, так как время работы исполнения программы упало в разы, а также можно отметить, что наименьший размер файла получился у MSVC с соответствующей оптимизацией.

Ещё один момент, который стоит отметить — это разный вывод программы, скомпилированной с оптимизациями и без. Это проявляется у компиляторов CLang и ICX. Вывод без оптимизации: “220”, а с оптимизациями:

“2201223Common subexpression elimination 000100007100072000730007400074000740007400074000740007000000”.

Причина этого — undefined behavior в коде, вызванном делением на ноль. Без оптимизаций программа “падает” на этом участке кода, в то время как с оптимизациями она старается продолжить выполнение. По стандарту C/C++ компилятор имеет право предполагать, что в корректной программе неопределённое поведение не наступает, поэтому он всячески преобразовывает код, чтобы избежать UB. Поэтому исходная программа была переписана, чтобы избежать данного поведения.

4. Заключение

Все три компилятора успешно справляются с базовыми оптимизациями: размножение и свёртка констант, арифметические тождества, лишние присваивания и тому подобное. Однако между ними есть различия.

ICX в среднем демонстрирует наиболее продвинутые техники оптимизации, ориентированные на максимальную скорость выполнения кода, часто в ущерб его размеру:

- Активно заменяет дорогие операции (деление idiv, умножение) на дешёвые (сдвиги shl, умножение на магические константы imul)
- Полностью разворачивает циклы, превращая их в линейный код, что устраняет накладные расходы на проверку условий, но сильно увеличивает размер бинарника
- Лучшим образом вычисляет константы на этапе компиляции, избавляясь от лишних вычислений во время выполнения

CLang во многом похож на ICX в базовых оптимизациях. Он также заменяет операции на более дешёвые (пример с умножением) и эффективно использует регистры, минимизируя обращения к памяти. Главное отличие между ними — это то, что CLang оставляет циклы, в то время как ICX полностью их разворачивает. CLang сохраняет баланс между производительностью и размером кода.

MSVC часто генерирует код, который легче отлаживать, но который может уступать в производительности:

- Включает проверки переполнения стека (__report_rangecheckfailure), а также инструкции int 3 и чаще использует безопасные, но более медленные инструкции
- Чаще обращается к памяти (dword ptr [...]) даже тогда, когда данные можно было бы держать в регистрах (в отличие от ICX)
- Реже применяет полное развёртывание циклов или сложную подстановку констант, оставляя больше вычислений на этап выполнения
- В некоторых примерах использует прямое деление (idiv), которое медленнее эмуляции через умножение